

Лабораторная работа №7.

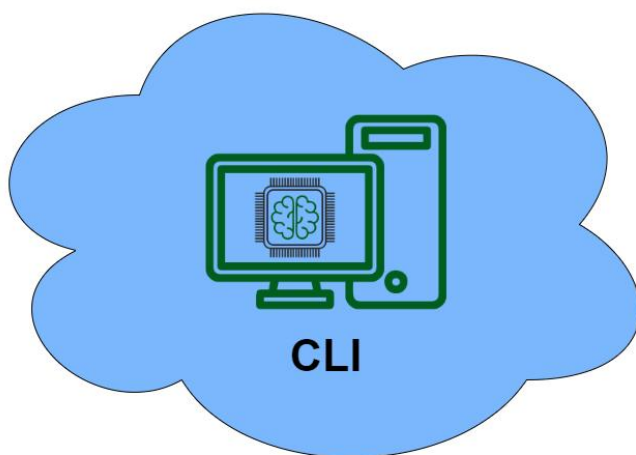
Управляющие конструкции командного интерпретатора

Цель работы:

- Приобрести практический опыт работы с перенаправлением потоков, gui и cli интерфейсами, основой написания shell скриптов и символами экранирования.

Оборудование, ПО:

- ALT1
- Справочная литература или доступ в сеть интернет.



Машина	IP	Маска	Шлюз	DNS
CLI (Альт Рабочая станция 10.4)	- 192.168.1.1	- 24	- -	- 77.88.8.8

Порядок работы:

1. На VM выполните команду
hostnamectl sethostname "Ваше ФИО"; exec bash
2. Ознакомиться с лабораторной работой. В поле "Результат работы" вставить скриншот работы скрипта. К каждому исполненному сценарию написать описание, что он делает.
3. Выполнить контрольные задания.

Контрольные задания:

```
#!/bin/bash
user=$1
if grep $user /etc/passwd
then
echo "Пользователь $user существует"
else
echo "Пользователь $user не существует"
fi
```

1. Доработайте данный скрипт, который будет искать пользователей “точно”, а не просто совпадения.
2. По каждой теме придумайте свой пример скрипта. С логикой и на практике применимый.
3. Напишите скрипт которому будут передаваться числа, а он будет проверять четность каждого из них. Если четных чисел больше на вывод сообщение “Четных больше”. Если нечетных чисел больше, на вывод текст “Нечетных больше”.
4. ***Напишите скрипт, который выводит информацию о текущем количестве пользователей в системе, дате и времени. Если время больше 3-х и время больше 12:00 скрипт должен также об этом уведомлять.
5. Разработайте скрипт, который считывает все числа, введенные пользователем, и каждую из них записывает в переменную. Далее четные переменные перемножаются, а нечетные суммируются, а 3 и 6 переменные, если они есть, не учитываются при выполнении расчетов.
6. Создайте ассоциативный массив, для записи информации о студентах. Параметры: ФИО, дата рождения, группа, институт. Наполните его данными. Сделайте запрос, который будет выводить только студентов чей возраст больше 21 года.
7. Напишите функцию, которая вычисляет наименьшее общее кратное двух чисел.
8. *** Разработать скрипт file-finder.sh, который находит в указанном при запуске (в качестве первого аргумента скрипта) каталоге все файлы указанного типа (например, файлы в формате ASCII.) Скрипт должен сохранять полученный список в файл, ТИП.txt, где ТИП - это значение, указанное в качестве второго аргумента скрипта. Проверку типа файла следует выполнять через не по расширению файла, а по фактическому типу, получаемому утилитой file. Тип следует проверять без учёта регистра в названии типа.
- Примеры запуска скрипта:
\$./file-finder.sh /home/vasya ascii
\$./file-finder.sh /home/sveta png
- 9.

Используемые источники:

- Официальная документация операционной системы Альт Рабочая Станция 10.4 <https://docs.altlinux.org/ru-RU/alt-workstation/10.4/html/alt-workstation/index.html>

Shell скрипты

Сценарии командной строки — наборы тех же самых команд, которые можно вводить с клавиатуры, собранные в файлы и объединённые некоей общей целью. При этом результаты работы команд могут представлять либо самостоятельную ценность, либо служить входными данными для других команд.

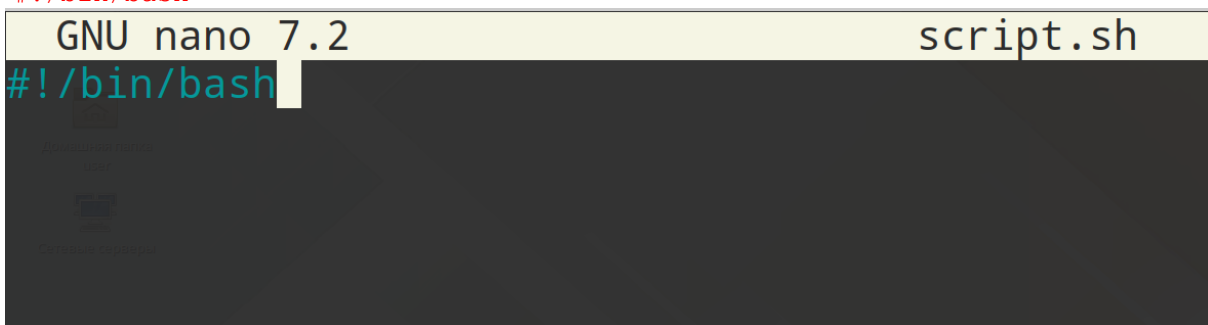
Сценарии — мощный способ автоматизации часто выполняемых действий. Итак, если говорить о командной строке, она позволяет выполнить несколько команд за один раз, введя их через точку с запятой:

Создадим пустой файл с использованием команды `touch`. В его первой строке нужно указать, какую именно оболочку мы собираемся использовать. Нас интересует `bash`, поэтому первая строка файла будет такой:

```
#touch script.sh
```

```
[user@alt test]$ touch script.sh
```

```
#!/bin/bash
```

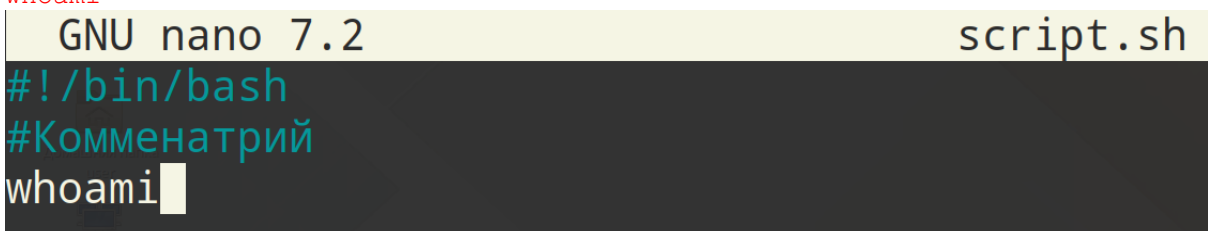


The screenshot shows a terminal window with the title bar "GNU nano 7.2" and "script.sh". The first line of the file is `#!/bin/bash`, followed by a blank line and a cursor.

В других строках этого файла символ решётки используется для обозначения комментариев, которые оболочка не обрабатывает. Однако, первая строка — особый случай, здесь решётка, за которой следует восклицательный знак (эту последовательность называют [шебанг](#)) и путь к `bash`, указывают системе на то, что сценарий создан именно для `bash`.

Команды оболочки отделяются знаком перевода строки, комментарии выделяют знаком решётки. Вот как это выглядит:

```
#!/bin/bash
#Комментарий
whoami
```



The screenshot shows the same terminal window as before, but now with three lines of code: `#!/bin/bash`, `#Комментарий`, and `whoami`. The cursor is at the end of the third line.

Тут, так же, как и в командной строке, можно записывать команды в одной строке, разделяя точкой с запятой. Однако, если писать команды на разных строках, файл легче читать. В любом случае оболочка их обработает.

Установка разрешений для файла сценария

Сохраните файл, дав ему имя `myscript`, и работа по созданию `bash`-скрипта почти закончена. Сейчас осталось лишь сделать этот файл исполняемым, иначе, попытавшись его запустить, вы столкнётесь с ошибкой `Permission denied`.

Сделаем файл исполняемым:

```
chmod +x ./script.sh
```

```
[user@alt test]$ chmod +x script.sh
```

Теперь попытаемся его выполнить:

```
./script.sh
```

```
[user@alt test]$ ./script.sh  
user
```

Вывод сообщений

Для вывода текста в консоль Linux применяется команда `echo`. Воспользуемся знанием этого факта и отредактируем наш скрипт, добавив пояснения к данным, которые выводят уже имеющиеся в нём команды:

```
#!/bin/bash  
# Тут вы можете писать комментарии  
echo "Текущая рабочая директория:"  
pwd  
echo "Залогиненный пользователь"  
whoami
```

```
[user@cli Документы]$ ./script.sh  
Текущая рабочая директория:  
/home/user/Документы  
Залогиненный пользователь  
user
```

Пользовательские переменные и переменные окружения

bash-скрипты позволяют задавать и использовать в сценарии собственные переменные. Подобные переменные хранят значение до тех пор, пока не завершится выполнение сценария.

- переменная=значение

Name="Igor"

Number=100

```
#!/bin/bash
Name=Igor
Number=100
echo $Name
echo $Number
```

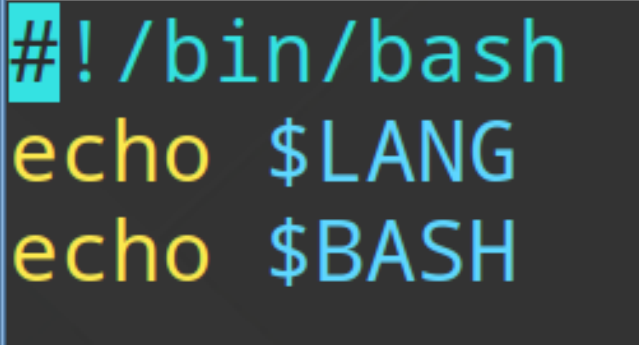
Результат работы

Также переменные можно переопределить:

```
#!/bin/bash
Number=100
echo $Number
Number=0
echo $Number
```

Результат работы

Также внутри тела скрипта можно обращаться к переменным среды



```
#!/bin/bash
echo $LANG
echo $BASH
```

Результат работы

Подстановка команд

Одна из самых полезных возможностей bash-скриптов — это возможность извлекать информацию из вывода команд и назначать её переменным, что позволяет использовать эту информацию где угодно в файле сценария.

Сделать это можно двумя способами.

- С помощью значка обратного апострофа `
- С помощью конструкции `$()`

```
mydir=`pwd`
```

```
user=$(whoami)
```

Итоговый скрипт:

```
#!/bin/bash
mydir=`pwd`
user=$(whoami)
echo Текущая директория $mydir
echo Я $user
```

Результат работы

Математические операции

Для выполнения математических операций в файле скрипта можно использовать конструкцию вида

- `$(a+b)`

`var1=$(( 25 + 5 ))`

`var2=$(( $var1 * 3 ))`

```
#!/bin/bash
var1=$(( 25 + 5 ))
echo $var1
var2=$(( $var1 * 3 ))
echo $var2
```

Результат работы

Позиционные и специальные параметры

Оболочка `bash` назначает специальным переменным, называемым позиционными параметрами, введенные при вызове скрипта параметры командной строки:

`$0` имя скрипта

`$1 - $9` первый-девятый параметр

`${10}`, `${11}`, ... десятый, одиннадцатый параметр

Специальные переменные командного интерпретатора

`$#` — Количество позиционных параметров, переданных сценарию.

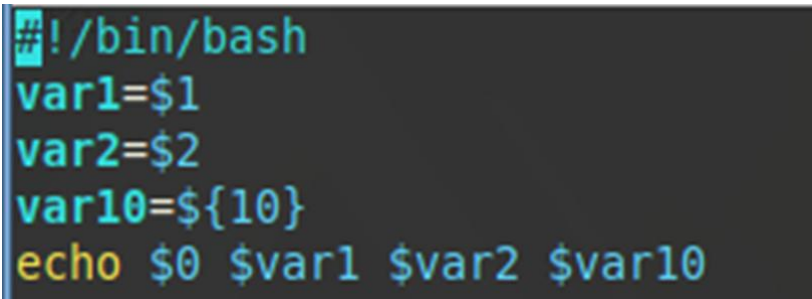
`$*` — Позиционные параметры, начиная с первого. Когда `$*` указывается в двойных кавычках, значение переменной заменяется одним словом, содержащим значения каждого параметра, разделенных первым символом специальной переменной `IFS`.

`$@` — Позиционные параметры, начиная с первого. Когда `$@` указывается в двойных кавычках, то каждый параметр возвращается в отдельном слове.

`$?` — Код возврата последней команды.

`!` — Возвращает идентификатор процесса последней команды, выполненной в фоновом режиме.

`$$` — Номер процесса, под которым выполняется данный сценарий.



```
#!/bin/bash
var1=$1
var2=$2
var10=${10}
echo $0 $var1 $var2 $var10
```

(при запуске скрипта передайте ему параметры `./script.sh 421 Привет 3 53 421`)

Результат работы



Когда вы используете эту команду, она, по умолчанию, сдвигает значения позиционных параметров влево. Например, значение переменной \$3 становится значением переменной \$2, значение \$2 переходит в \$1, а то, что было до этого в \$1, теряется. Обратите внимание на то, что при этом значение переменной \$0, содержащей имя скрипта, не меняется.

(в качестве переменных передайте скрипту числа от 0 до 10)

--

```
#!/bin/bash
ls non_existent_file
echo "Exit status: $?"
```

Результат работы

```
#!/bin/bash
ls /home
echo "Exit status: $?"
```

Результат работы

Управляющая конструкция if-then-else

Для того, чтобы программа смогла сообщить и о результатах успешного поиска, и о неудаче, существует конструкция if-then-else.

Синтаксис

if команда

then

команды

fi

или

if команда

then

команды

else

КОМАНДЫ

fi

Если первая команда возвратит ноль, что означает её успешное выполнение, условие окажется истинным и выполнение не пойдет по ветке else. В противном случае, если будет возвращено что-то, отличающееся от нуля, что будет означать неудачу, или ложный результат, будут выполнены команды, расположенные после else.

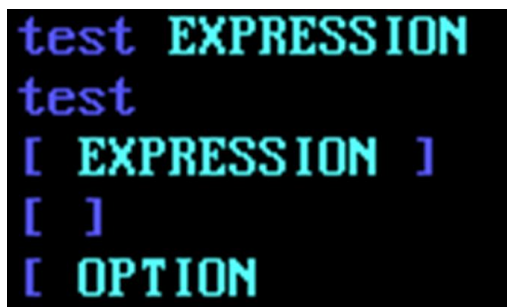
```
#!/bin/bash
user=$1
if grep $user /etc/passwd
then
echo "Пользователь $user существует"
else
echo "Пользователь $user не существует"
fi
```

Результат работы

Команда test

Предназначена для проверки типа файла и сравнения чисел и строк. Возвращает код возврата 0 (истина) или 1 (ложь) в зависимости от вычисления выражения. Выражения могут быть как унарными, так и бинарными.

Синтаксис



```
test EXPRESSION
test
[ EXPRESSION ]
[ ]
[ OPTION
```

- d файл — истина, если file существует и является каталогом
- e файл — истина, если file существует
- f файл — истина, если file существует и является обычным файлом
- r файл — истина, если file существует и доступен для чтения
- s файл — истина, если file существует и имеет размер больше, чем ноль
- w файл — истина, если file существует и доступен для записи
- x файл — истина, если file существует и доступен для выполнения
- L файл — истина, если file существует и является символьной ссылкой
- файл1 -nt файл2 — истина, если file1 новее (дата модификации), чем file2
- файл1 -ot файл2 — истина, если file1 старше, чем file2
- z строка — истина, если длина string равна нулю
- n строка — истина, если длина string не ноль
- строка1 = строка2 — истина, если строки идентичны
- строка1 != строка2 — истина, если строки не идентичны
- str1 < str2 Возвращает истину, если str1 меньше, чем str2
- str1 > str2 Возвращает истину, если str1 больше, чем str2.
- число1 -eq число2 — истина, если число1 равно число2
- число1 -gt число2 — истина, если число1 больше число2
- число1 -ge число2 — истина, если число1 больше или равно число2
- число1 -lt число2 — истина, если число1 меньше число2
- число1 -le число2 — истина, если число1 меньше или равно число2
- число1 -ne число2 — истина, если число1 не равно число2
- expr1 -a expr2 — истина, если оба выражения expr1 и expr2 истина (операция AND)
- expr1 -o expr2 — истина, если хотя бы одно из выражений expr1 или expr2 истина (операция OR)
- ! expr — истина, если выражение expr ложно

```
var1=10
var2=5
var3='Десять'
var4='Пять'
var5=/root/script.sh
```

```
if [ -d $var5 ]
then echo 'Это каталог'
else echo 'Это файл'
fi
```

Результат работы

```
if [ $var1 -ge $var2 ]
then echo "Число $var1 больше $var2"
else echo "Число $var1 меньше $var2"
fi
```

Результат работы

```
var3 /root/script.sh
if [ $var3 = $var4 ]
then echo "Строка -$var3- равна строке -$var4-"
else echo "Строка -$var3- не равна строке -$var4-"
fi
```

Результат работы

```
var3 /root/script.sh
if test $var3 = $var4
then echo "Строка -$var3- равна строке -$var4-"
else echo "Строка -$var3- не равна строке -$var4-"
fi
```

Результат работы

```
#!/bin/bash
var1=слово
var2=Слово
if [ $var1 > $var2 ]
then
echo "$var1 is greater than $var2"
else
echo "$var1 is less than $var2"
fi
```

Результат работы

Циклы for

Оболочка bash поддерживает циклы for, которые позволяют организовывать перебор последовательностей значений.

Синтаксис

for var in list

do

команды

done

В каждой итерации цикла в переменную var будет записываться следующее значение из списка list. В первом проходе цикла, таким образом, будет задействовано первое значение из списка. Во втором — второе, и так далее — до тех пор, пока цикл не дойдёт до последнего элемента.

```
#!/bin/bash
for var in 1 2 3 4 5 6 7 8
do
echo Цифра $var
done
```

Результат работы

```
#!/bin/bash
iter=0
for var in {0..1000..20}
do
echo Текущая итерация $iter
iter=$(( $iter + 1 ))
echo Число в цикле $var
done
```


Результат работы
<pre>#!/bin/bash for file in /home/user/* do if [-d "\$file"] then echo "\$file is a directory" elif [-f "\$file"] then echo "\$file is a file" fi done</pre>

Результат работы

```
#!/bin/bash
for line in $(cat file.txt);
do
echo $line
done
```

Результат работы

Циклы for в стиле C

Если вы знакомы с языком программирования C, синтаксис описания bash-циклов for может показаться вам странным, так как привыкли вы, очевидно, к такому описанию циклов:

```
for (i = 0; i < 10; i++)
```

```
#!/bin/bash
for (( i=1; i <= 10; i++ ))
do
echo "number is $i"
done
```

Результат работы

Получение данных от пользователя

Ключи и параметры командной строки — это отличный способ получить данные от того, кто пользуется скриптом, однако в некоторых случаях нужно больше интерактивности.

Иногда сценарии нуждаются в данных, которые пользователь должен ввести во время выполнения программы. Именно для этой цели в оболочке bash имеется команда read.

Эта команда позволяет принимать введенные данные либо со стандартного ввода (с клавиатуры), либо используя другие дескрипторы файлов. После получения данных, эта команда помещает их в переменную:

```
echo -n "Введите свое имя: "  
read name  
echo "Пользователь $name, добро пожаловать в скрипт."
```

Описание скрипта:

Результат работы:

При вызове read можно указывать и несколько переменных:

```
echo -n "Введите свое Фамилию Имя и Дату Рождения: "  
read name surname birth  
echo "Пользователь, $name $surname $birth года рождения, добро пожаловать в скрипт."
```

Описание скрипта:

Результат работы:

Циклы While

Цикл while используется для выполнения блока кода до тех пор, пока заданное условие истинно. Основной синтаксис цикла while в оболочке Bash выглядит следующим образом:

while [условие]; do

—код—

done

В этом синтаксисе условие должно быть помещено внутри квадратных скобок ([]), ключевое слово `do` отмечает начало блока кода, который будет выполняться, а ключевое слово `done` - конец блока кода.

Вот несколько примеров того, как можно использовать циклы `while` в оболочке Bash:

```
#!/bin/bash
counter=0
while [ $counter -lt 5 ]; do
    echo $counter
    counter=$((counter + 1))
done
```

Описание скрипта:
Результат работы:

```
#!/bin/bash
while ! ping -c 1 $1; do
    echo " Не пингуется ((( "
```

Описание скрипта:

Результат работы:

Команды управления циклами break и continue

break - прерывает выполнение цикла, управление переходит на следующий оператор

continue - прерывает выполнение очередной итерации тела цикла

```
#!/bin/bash
for var1 in {1..10};
do
if [ $var1 -eq 5 ]; then
break
fi
echo "Число: $var1"
done
```

Описание скрипта:

Результат работы:

```
#!/bin/bash
for (( var1 = 1; var1 < 15; var1++ ));
do
if [ $var1 -gt 5 ] && [ $var1 -lt 10 ];
then
continue
fi
echo "Номер итерации: $var1"
done
```

Описание скрипта:
Результат работы:

Массивы

Помимо скалярных переменных, которые хранят одно значение, переменные оболочки могут быть массивами, которые хранят несколько значений, и ассоциативными массивами, которые хранят пары ключ-значение. Чтобы объявить массив, вы используете команду `declare` или `typeset` с параметром `-a`. Например, следующая команда объявляет массив с именем `colors` с тремя элементами:

```
declare -a colors  
colors=(red green blue)
```

Для доступа к элементу массива используется оператор `[]` с индексом элемента. Например, следующая команда выводит первый элемент массива `colors`:

```
echo ${colors[0]}
```

Описание скрипта:
Результат работы:

Чтобы объявить ассоциативный массив, вы используете команду `declare` или `typeset` с опцией `-A`. Например, следующая команда объявляет ассоциативный массив с именем `person` с двумя элементами:


```
declare -A person
person=([name]=Alblak [age]=52)
```

Для доступа к элементу ассоциативного массива используется оператор `[]` с ключом элемента. Например, следующая команда выводит значение ключа `name` в ассоциативном массиве `person`:

```
echo ${person[name]}
```

Описание скрипта:

Результат работы:

Функции

Занимаясь разработкой `bash`-скриптов, вы рано или поздно столкнётесь с тем, что вам периодически приходится использовать одни и те же фрагменты кода. Постоянно набирать их вручную скучно, а копирование и вставка — не наш метод. Как быть? Хорошо бы найти средство, которое позволяет один раз написать блок кода и, когда он понадобится снова, просто сослаться на него в скрипте.

Оболочка `bash` предоставляет такую возможность, позволяя создавать функции. Функции `bash` — это именованные блоки кода, которые можно повторно использовать в скриптах.

Синтаксис:

```
functionName {
КОД
}
или
functionName() {
КОД
}
```

Команда `return` позволяет задавать возвращаемый функцией целочисленный код завершения. Есть два способа работы с тем, что является результатом вызова функции.

```
function myfunc {
  read -p "Введите число, которое хотите возвести в квадрат: " value
  echo ""
  return $(( $value * $value ))
}

myfunc

echo "Полученное значение $?"
```

Описание скрипта:

Результат работы:

Если вы выполните любую другую команду до извлечения из переменной `$?` значения, возвращаемого функцией, это значение будет утеряно. Дело в том, что данная переменная хранит код возврата последней выполненной команды.

Учтите, что максимальное число, которое может вернуть команда `return` — 255. Если функция должна возвращать большее число или строку, понадобится другой подход.

Более корректный способ возврата результатов работы функции заключается в записи данных, выводимых функцией, в переменную. Такой подход позволяет обойти ограничения команды `return` и возвращать из функции любые данные.

```
function myfunc {
  read -p "Введите число, которое хотите возвести в квадрат: " value
  value=$(( $value * $value ))
  echo $value
}

result=$(myfunc)

echo "Полученное значение $result"
```

Описание скрипта:
Результат работы:

Рекурсивные функции

Рекурсия — это когда функция сама себя вызывает. Классический пример рекурсии — функция для вычисления факториала. Факториал числа — это произведение всех натуральных чисел от 1 до этого числа. Например, факториал 5 можно найти так:

$$5! = 1 * 2 * 3 * 4 * 5$$

```
#!/bin/bash
function factorial {
if [ $1 -eq 1 ]
then
echo 1
else
local temp=$(( $1 - 1 ))
local result=$(factorial $temp)
echo $(( $result * $1 ))
fi
}

read -p "Введите число: " value
result=$(factorial $value)
echo "Факториал $value равен: $result"
```

Описание скрипта:

Результат работы:

Ключи командной строки

Ключи командной строки обычно выглядят как буквы, перед которыми ставится тире. Они служат для управления сценариями. Рассмотрим такой пример:

```
#!/bin/bash
echo
while [ -n "$1" ]
do
case "$1" in
-a) echo "Найдена -a опция" ;;
-b) echo "Найдена -b опция" ;;
-c) echo "Найдена -c опция" ;;
*) echo "$1 не опция" ;;

esac
shift
done
```

Описание скрипта:

В этом коде использована конструкция case, которая сверяет переданный ей ключ со списком обрабатываемых скриптом ключей. Если переданное значение нашлось в этом списке, выполняется соответствующая ветвь кода. Если при вызове скрипта будет использован любой ключ, обработка которого не предусмотрена, будет исполнена ветвь «*»

Результат работы:

Часто при написании `bash`-скриптов возникает ситуация, когда надо использовать и параметры командной строки, и ключи. Стандартный способ это сделать заключается в применении специальной последовательности символов, которая сообщает скрипту о том, когда заканчиваются ключи и начинаются обычные параметры.

Эта последовательность — двойное тире (`--`). Оболочка использует её для указания позиции, на которой заканчивается список ключей. После того, как скрипт обнаружит признак окончания ключей, то, что осталось, можно, не опасаясь ошибок, обрабатывать как параметры, а не как ключи.

Скрипт, указанный ниже, использует команду `break` для прерывания цикла `while` при обнаружении в строке двойного тире.

```
#!/bin/bash
while [ -n "$1" ]
do
case "$1" in

-a) echo "Найдена -a опция" ;;
-b) echo "Найдена -b опция";;
-c) echo "Найдена -c опция" ;;
--) shift
break ;;
*) echo "$1 не опция";;

esac
shift
done
count=1
for param in $@
do
echo "Параметр #$count: $param"
count=$(( $count + 1 ))
done
```

Описание скрипта:

Результат работы: